AI Engineer — Take-Home Assignment

Video Surveillance: Detection, Tracking & Event Recognition

Overview

Build a system that processes security camera footage to detect people, track them across frames, and identify events of interest. This assignment evaluates your ability to design a practical computer vision pipeline, make informed model selection decisions, and think about production-readiness.

Time Budget: 8–10 hours
Deliverables: Working code repository, annotated sample output, and a README covering architecture decisions, model choices, and known limitations.

Sample Video Sources

You may use clips from any of the following **free, publicly available** datasets. We recommend selecting **3–5 clips** (30–60 seconds each) covering a mix of indoor/outdoor scenes, varying lighting, and crowd densities.

Dataset	What It Contains	Download
VIRAT	Ground & aerial surveillance of pedestrians and vehicles in outdoor settings. Multiple resolutions (HD + downsampled). Activity annotations included.	viratdata.org
UCF-Crime	1,900 real-world untrimmed CCTV videos (~128 hrs). 13 anomaly categories (fighting, robbery, loitering, etc.) plus normal activity. ~320×240, 25–30 fps.	UCF CRCV Project Page or Kaggle Mirror
MOT17	Multi-object tracking benchmark. Pedestrian-focused sequences with ground-truth bounding boxes and track IDs. Ideal for evaluating tracking accuracy.	motchallenge.net/data/MOT17
MEVA / MEVADATA	250+ hours of multi-camera ground video with UAV footage. Activity-annotated subset of 12.5 hours. Extension of VIRAT under IARPA's DIVA program.	mevadata.org
VisDrone	Drone-captured sequences with pedestrian, car, bus, truck, and van annotations. Good for testing multi-scale detection.	github.com/VisDrone

> **Tip:** For a manageable scope, pick 2–3 short clips from **MOT17** (clean ground truth for tracking evaluation) and 1–2 from **UCF-Crime** (real-world messiness, varied quality).

Requirements

Core (Must Have)

1. **Person Detection & Tracking**
   - Detect and track people across frames with unique IDs
   - Handle re-identification when a person temporarily leaves and re-enters the frame
   - Output bounding boxes and confidence scores per frame
2. **Zone-Based Event Detection**
   - Define zones of interest via a config file (JSON with polygon coordinates)
   - Detect at least two event types:

    - **Zone intrusion** — person enters a defined restricted area
    - **Loitering** — person remains stationary in a zone beyond a configurable time threshold
  - Generate timestamped event logs with frame number, bounding box, event type, and confidence
3. **Output Generation**
  - Produce an annotated video or image sequence showing detections, tracks, and zone overlays
  - Generate a structured event log (JSON or CSV) summarizing all detected events with timestamps
4. **Interface**
  - Provide a CLI or simple API to run the pipeline:

```
python run.py --video input.mp4 --zones zones.json --output results/
```

Stretch Goals (Nice to Have)

- Real-time or near-real-time processing with FPS benchmarks
- Dashboard or web UI showing detections live
- Multi-camera / multi-video support
- Configurable alert thresholds and deduplication logic
- Evaluation script comparing detections against ground-truth annotations (e.g., MOTA/MOTP metrics)

Evaluation Criteria

Area	What We're Looking For	Weight
Model Selection	Justified choice of detection model (YOLO, Faster R-CNN, etc.) and tracker (DeepSORT, ByteTrack, etc.). Understanding of trade-offs: speed vs accuracy, pre-trained vs fine-tuned.	20%
Pipeline Architecture	Clean separation of concerns — detection, tracking, event logic, output. Modular and extensible design. Frame sampling and batching strategy.	25%

Area	What We're Looking For	Weight
Event Detection Logic	Robust zone intersection math (point-in-polygon). Sensible temporal reasoning for loitering. State management across frames. Handling of edge cases.	20%
Edge Case Handling	How the system handles: occlusion, ID switches, lighting changes, camera shake, empty frames, crowded scenes.	15%
Production Thinking	GPU/CPU awareness, memory management on long videos, alert deduplication, logging, configuration management, reproducibility.	10%
Code Quality & Documentation	Clean, readable code. Clear README with setup instructions, architecture diagram, and discussion of trade-offs and limitations.	10%

What to Include in Your README

- **Architecture overview** — diagram or description of the pipeline stages
- **Model choices** — which models you selected and why (include alternatives you considered)
- **Setup instructions** — how to install dependencies and run the pipeline
- **Configuration** — how to define zones, adjust thresholds, and modify parameters
- **Sample results** — screenshots or GIFs of annotated output
- **Known limitations** — what breaks, what you'd improve with more time
- **Performance notes** — FPS, memory usage, hardware used

Submission

- Share a Git repository (GitHub, GitLab, or Bitbucket)
- Include a `requirements.txt` or `Dockerfile` for reproducibility
- Include sample output files (annotated video clips + event logs)
- Ensure the pipeline runs end-to-end with a single command

What We're NOT Looking For

- Perfect accuracy — we care more about your approach than benchmark numbers

- A production-deployed system — a well-structured prototype is sufficient
- Novel research — use existing models and libraries; show good engineering judgment

---

Good luck. We look forward to reviewing your work.